

# EDA Final - Soluciones Teoricas Racso

## Un problema por hoja, explicado desde cero

### Como estudiar este PDF

Este documento esta escrito para repasar desde cero los problemas que si sirven para el final: los concursos utiles del grupo Racso, mas los externos que refuerzan exactamente los mismos temas. No hay codigo. En cada problema debes aprender a escribir en papel: estructura, algoritmo, correctitud y complejidad.

Los temas que se repiten son:

- RMQ, LCA y consultas en arboles estaticos.
- Suffix array, LCP, busqueda y conteo de subcadenas.
- Euler tour, DSU on tree, HLD, mascarar de paridad, centroid decomposition.
- Dynamic connectivity offline con segment tree sobre tiempo y DSU rollback.
- Dynamic trees con Link-Cut Trees o Euler-Tour Trees.

No se incluyen Proyecto 2/Efecto cascada ni Proyecto 3/BST implicito porque no son prioridad para esta ruta.

### Contents

|           |                                     |           |
|-----------|-------------------------------------|-----------|
| <b>1</b>  | <b>Static RMQ</b>                   | <b>3</b>  |
| <b>2</b>  | <b>Suffix array y LCP array</b>     | <b>4</b>  |
| <b>3</b>  | <b>Busqueda de subcadenas</b>       | <b>5</b>  |
| <b>4</b>  | <b>Contando subcadenas</b>          | <b>6</b>  |
| <b>5</b>  | <b>Subcadenas diferentes</b>        | <b>7</b>  |
| <b>6</b>  | <b>Subcadena en comun mas larga</b> | <b>8</b>  |
| <b>7</b>  | <b>Ordenando subcadenas</b>         | <b>9</b>  |
| <b>8</b>  | <b>Bordes</b>                       | <b>10</b> |
| <b>9</b>  | <b>Refran</b>                       | <b>11</b> |
| <b>10</b> | <b>Substring periodica</b>          | <b>12</b> |
| <b>11</b> | <b>Tree and Queries</b>             | <b>13</b> |
| <b>12</b> | <b>Arpa's letter-marked tree</b>    | <b>14</b> |
| <b>13</b> | <b>Laminar Family</b>               | <b>15</b> |
| <b>14</b> | <b>Xenia and Tree</b>               | <b>16</b> |
| <b>15</b> | <b>Beauty and The Tree</b>          | <b>17</b> |

|                                                    |    |
|----------------------------------------------------|----|
| 16 Connect and Disconnect                          | 18 |
| 17 Addition on Segments                            | 19 |
| 18 Visitas                                         | 20 |
| 19 Distancia unitaria                              | 21 |
| 20 Imperial roads                                  | 22 |
| 21 Fools and Roads                                 | 23 |
| 22 Frequency of String                             | 24 |
| 23 Yosupo - Dynamic Graph Vertex Add Component Sum | 25 |
| 24 Yosupo - Dynamic Tree Subtree Add Subtree Sum   | 26 |
| 25 SPOJ - Dynamic Tree Connectivity                | 27 |
| 26 Resumen final                                   | 28 |

# 1 Static RMQ

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/694840/problem/A>

## Resumen

Se da un arreglo fijo  $A[0..n-1]$  y muchas consultas  $[l, r]$ . Para cada consulta se pide el minimo valor dentro del rango. El arreglo no cambia. Esa palabra, *static*, es lo que permite preprocesar fuerte.

## Teoria desde cero

RMQ significa *Range Minimum Query*. Si el arreglo cambiara, se usaria un segment tree. Pero aqui no cambia, entonces conviene guardar respuestas para rangos especiales. La estructura mas importante para empezar es **sparse table**. Guarda minimos de bloques de longitud potencia de dos:

$$st[k][i] = \min(A[i], A[i+1], \dots, A[i+2^k-1]).$$

El bloque de longitud  $2^k$  se parte en dos bloques de longitud  $2^{k-1}$ :

$$st[k][i] = \min(st[k-1][i], st[k-1][i+2^{k-1}]).$$

## Algoritmo

Preprocesa todos los  $st[k][i]$ . Para consultar  $[l, r]$ , toma  $len = r - l + 1$  y  $k = \lfloor \log_2 len \rfloor$ . Usa dos bloques de longitud  $2^k$ : uno empieza en  $l$  y el otro termina en  $r$ :

$$ans = \min(st[k][l], st[k][r - 2^k + 1]).$$

Los bloques pueden solaparse.

## Correctitud

Los dos bloques cubren todo el intervalo  $[l, r]$ . El minimo global del intervalo esta en al menos uno de los dos bloques. Si los bloques se solapan, no hay problema porque el minimo es idempotente:  $\min(x, x) = x$ . Por eso tomar el minimo entre los minimos de ambos bloques devuelve exactamente el minimo del rango.

## Complejidad

Preprocesamiento  $O(n \log n)$ , memoria  $O(n \log n)$  y cada consulta  $O(1)$ . En teoria avanzada existe RMQ con  $O(n)$  espacio y  $O(1)$  query usando macro/micro bloques, pero sparse table es la respuesta limpia para papel si no piden optimalidad estricta.

## 2 Suffix array y LCP array

Link. <https://codeforces.com/group/Amk1RC8lhZ/contest/695758/problem/A>

### Resumen

Este problema es la base de todo el bloque de strings: construir o razonar con el suffix array y el LCP array de una cadena.

### Teoria desde cero

Un sufijo de  $S$  es  $S[i..n-1]$ . El suffix array SA es una permutacion de posiciones tal que los sufijos quedan ordenados lexicograficamente:

$$S[SA[0]..] < S[SA[1]..] < \dots < S[SA[n-1]..].$$

El LCP array mide prefijos comunes entre sufijos consecutivos:

$$LCP[i] = \text{lcp}(S[SA[i-1]..], S[SA[i]..]).$$

Un caracter sentinela menor que todos, por ejemplo \$, evita problemas cuando un sufijo es prefijo de otro.

### Estructura y algoritmo

El algoritmo de clase mas importante es Manber–Myers. Mantiene clases de equivalencia para prefijos de longitud  $2^k$ . Inicialmente se ordena por primer caracter. En la ronda  $k$ , el sufijo que empieza en  $i$  se representa por:

$$(rank[i], rank[i + 2^k]).$$

Ordenar esos pares equivale a ordenar por los primeros  $2^{k+1}$  caracteres. Luego se reasignan ranks. Al llegar a longitud al menos  $n$ , los ranks ordenan sufijos completos.

### Correctitud

Si dos sufijos tienen iguales sus primeros  $2^{k+1}$  caracteres, entonces sus primeros  $2^k$  caracteres coinciden y tambien coinciden los siguientes  $2^k$ . Por eso el par de ranks describe correctamente el prefijo doble. Si los pares difieren, el orden lexicografico del par decide el orden del prefijo completo. Por induccion en  $k$ , cada ronda ordena por una longitud cada vez mayor, y al superar  $n$  queda el orden de sufijos.

### Complejidad

Hay  $\lceil \log n \rceil$  rondas. Si cada ronda se ordena con counting/radix sort sobre ranks, cuesta  $O(n)$  por ronda y  $O(n \log n)$  total. El LCP puede calcularse con Kasai en  $O(n)$  usando el rank inverso de cada sufijo.

### 3 Búsqueda de subcadenas

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/695758/problem/B>

#### Resumen

El objetivo típico es responder si un patrón  $P$  aparece en un texto fijo  $S$ , cuantas veces aparece, o listar sus posiciones.

#### Teoría desde cero

Un patrón aparece en  $S$  si existe un sufijo de  $S$  que empieza con  $P$ . En el suffix array, los sufijos que empiezan con el mismo prefijo forman un intervalo contiguo. Esta es la propiedad central.

#### Algoritmo

Construye SA de  $S$ . Para una consulta con patrón  $P$ :

1. Busca con binary search el primer sufijo que es lexicográficamente mayor o igual que  $P$ .
2. Busca el primer sufijo que ya es estrictamente mayor que cualquier cadena que tenga prefijo  $P$ .
3. El intervalo entre ambos límites contiene todas las ocurrencias.

Si el intervalo es vacío, el patrón no aparece. Si tiene tamaño  $t$ , aparece  $t$  veces. Las posiciones son los valores  $SA[i]$  dentro del intervalo.

#### Correctitud

Todos los strings que comparten un prefijo  $P$  aparecen juntos en orden lexicográfico: no puede haber entre ellos un sufijo que no tenga prefijo  $P$ , porque eso rompería el orden en el primer carácter donde difiere. Las dos búsquedas binarias encuentran exactamente los bordes inferior y superior de ese bloque. Por tanto, el intervalo es exactamente el conjunto de ocurrencias.

#### Complejidad

Construcción  $O(n \log n)$  con Manber–Myers. Cada comparación directa cuesta  $O(|P|)$ , así que query  $O(|P| \log n)$  en la versión simple. Con LCP y RMQ se puede mejorar, pero para teoría de examen basta explicar el intervalo.

## 4 Contando subcadenas

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/695758/problem/C>

### Resumen

El nombre puede apuntar a contar ocurrencias de patrones o contar subcadenas con alguna condicion. La base para resolverlo es entender que suffix array transforma problemas de subcadenas en intervalos de sufijos.

### Teoria desde cero

Toda subcadena de  $S$  es prefijo de algun sufijo. Si quieres contar ocurrencias de una subcadena/patron  $P$ , debes contar cuantos sufijos empiezan con  $P$ . En SA, eso es el tamaño de un intervalo contiguo.

### Algoritmo para contar ocurrencias

Construye SA. Para cada patron  $P$ , encuentra el intervalo  $[L, R]$  de sufijos que tienen prefijo  $P$  con dos busquedas binarias. La cantidad de ocurrencias es:

$$R - L + 1.$$

Si el intervalo no existe, la respuesta es 0.

### Si la pregunta es contar subcadenas distintas

Entonces se usa la formula del problema siguiente:

$$\frac{n(n+1)}{2} - \sum_i \text{LCP}[i].$$

La diferencia es: contar ocurrencias de un patron usa intervalos; contar subcadenas distintas usa el total menos duplicados medidos por LCP.

### Complejidad

Preprocesar SA cuesta  $O(n \log n)$  o  $O(n)$  con DC3/SA-IS. Cada consulta de conteo cuesta  $O(|P| \log n)$  en la version simple. La memoria es  $O(n)$  para SA y ranks, mas lo necesario para LCP si se usa.

## 5 Subcadenas diferentes

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/695758/problem/D>

### Resumen

Se pide contar cuantas subcadenas distintas tiene una cadena.

### Teoria desde cero

Un string de longitud  $n$  tiene  $\frac{n(n+1)}{2}$  subcadenas contando repetidas. Cada subcadena es un prefijo de algun sufijo. Si procesas sufijos en orden lexicografico, el sufijo actual aporta nuevos prefijos salvo los que ya compartia con el sufijo anterior.

### Formula

El sufijo que empieza en  $SA[i]$  tiene longitud  $n - SA[i]$ , asi que tiene  $n - SA[i]$  prefijos. Pero sus primeros  $LCP[i]$  prefijos ya aparecieron como prefijos del sufijo anterior en orden lexicografico. Entonces aporta:

$$(n - SA[i]) - LCP[i].$$

Sumando sobre todos los sufijos:

$$\#distinct = \frac{n(n+1)}{2} - \sum_i LCP[i].$$

### Correctitud

Cada subcadena se cuenta cuando aparece por primera vez como prefijo de algun sufijo en orden lexicografico. Si un prefijo del sufijo actual ya aparecio antes, entonces aparece en algun sufijo vecino con prefijo comun; el LCP con el sufijo anterior captura exactamente la cantidad de prefijos que no son nuevos. Por eso restar la suma de LCP elimina duplicados una sola vez.

### Complejidad

Construir SA y LCP domina el costo:  $O(n \log n)$  con Manber–Myers y Kasai, o  $O(n)$  con construccion lineal. El conteo final es  $O(n)$ .

## 6 Subcadena en comun mas larga

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/695758/problem/E>

### Resumen

Dadas dos cadenas  $A$  y  $B$ , se pide la longitud o contenido de la subcadena mas larga que aparece en ambas.

### Teoria desde cero

Si una cadena  $X$  aparece en  $A$  y en  $B$ , entonces existen dos sufijos, uno de  $A$  y uno de  $B$ , que tienen a  $X$  como prefijo comun. El problema se vuelve buscar el mayor LCP entre sufijos de origen distinto.

### Algoritmo

Concatena:

$$T = A\#B\$,$$

donde  $\#$  y  $\$$  no aparecen en las cadenas. Construye SA y LCP de  $T$ . Recorre posiciones consecutivas del suffix array. Si  $SA[i-1]$  viene de  $A$  y  $SA[i]$  viene de  $B$ , o viceversa, actualiza la respuesta con  $LCP[i]$ .

### Correctitud

Todos los sufijos que comparten un prefijo  $X$  aparecen consecutivos en SA. Si en ese bloque hay al menos un sufijo de  $A$  y uno de  $B$ , entonces dentro del bloque existe algun par adyacente de origen distinto. El LCP de ese par mide una subcadena comun de longitud al menos  $|X|$ . Por tanto el maximo entre pares adyacentes de origen distinto da la subcadena comun mas larga.

### Complejidad

Construccion de SA y LCP:  $O(n \log n)$  o  $O(n)$  segun algoritmo. Recorrido final:  $O(n)$ . Memoria  $O(n)$ .



## 7 Ordenando subcadenas

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/695758/problem/F>

### Resumen

Se deben comparar u ordenar subcadenas de un mismo texto. El problema combina suffix array, LCP y RMQ.

### Teoria desde cero

Para comparar  $S[l_1..r_1]$  y  $S[l_2..r_2]$ , necesitas saber el primer punto donde difieren. Eso es el LCP entre los sufijos  $S[l_1..]$  y  $S[l_2..]$ , recortado por las longitudes de las subcadenas.

### Estructura

Construye:

- suffix array SA;
- rank inverso: posicion de cada sufijo en SA;
- LCP array;
- RMQ sobre LCP para obtener LCP entre dos sufijos arbitrarios.

Si los sufijos de  $l_1$  y  $l_2$  estan en posiciones  $a < b$  en SA:

$$\text{lcp}(l_1, l_2) = \min(\text{LCP}[a + 1], \dots, \text{LCP}[b]).$$

### Comparador teorico

Sea  $x$  el LCP de los sufijos y  $len_1, len_2$  las longitudes de las subcadenas. Si  $x \geq \min(len_1, len_2)$ , entonces una subcadena es prefijo de la otra y decide la longitud. Si no, se compara  $S[l_1 + x]$  con  $S[l_2 + x]$ .

### Complejidad

Preprocesamiento  $O(n \log n)$  con sparse table sobre LCP. Cada comparacion  $O(1)$ . Ordenar  $q$  subcadenas cuesta  $O(q \log q)$  comparaciones.

## 8 Bordes

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/695758/problem/G>

### Resumen

Un borde de una cadena es una cadena que es prefijo y sufijo a la vez. Por ejemplo, en **ababa**, **a** y **aba** son bordes.

### Teoria desde cero

La herramienta mas directa es la funcion prefijo de KMP. Para cada posicion  $i$ :

$$\pi[i] = \text{longitud del mayor borde de } S[0..i].$$

Si conoces el mayor borde de todo el string, los bordes menores se obtienen siguiendo bordes del borde:

$$k = \pi[n - 1], \quad k = \pi[k - 1], \quad \dots$$

### Algoritmo

Calcula  $\pi$ . Empieza con  $k = \pi[n - 1]$ . Mientras  $k > 0$ , registra  $k$  como una longitud de borde y reemplaza  $k$  por  $\pi[k - 1]$ . Al final, si el problema pide tambien el string completo como borde impropio/propio, se maneja segun definicion.

### Correctitud

Si  $X$  es borde de  $S$ , entonces  $X$  es prefijo de  $S$  y tambien sufijo. El mayor borde esta en  $\pi[n - 1]$ . Cualquier borde menor de  $S$  debe ser borde de ese mayor borde; por eso seguir la cadena de  $\pi$  enumera todos los bordes.

### Complejidad

KMP prefix function cuesta  $O(n)$ . Enumerar bordes cuesta proporcional al numero de bordes, a lo mucho  $O(n)$ .

## 9 Refran

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/695758/problem/H>

### Resumen

Este tipo de problema suele pedir una subcadena repetida que maximiza alguna funcion, por ejemplo longitud por frecuencia. La estructura natural es suffix array + LCP.

### Teoria desde cero

Una subcadena aparece  $k$  veces si existen  $k$  sufijos que tienen esa subcadena como prefijo. En el suffix array, esos  $k$  sufijos aparecen en un bloque consecutivo. La longitud comun minima del bloque se mide por el minimo de los LCP internos.

### Estructura

Interpreta el arreglo LCP como un histograma. Una altura  $h$  mantenida sobre un intervalo de LCP significa que hay varios sufijos consecutivos que comparten un prefijo de longitud al menos  $h$ . Si el intervalo tiene  $w$  barras de LCP, cubre  $w + 1$  sufijos, entonces la frecuencia es  $w + 1$ .

### Algoritmo conceptual

Usa pila monotonamente decreciente sobre LCP para cada posicion como altura minima de un intervalo maximal. Para cada altura  $h$ , calcula cuantos sufijos consecutivos tienen LCP al menos  $h$ , y evalua la funcion pedida, por ejemplo:

$$h \cdot (\text{cantidad de sufijos}).$$

### Complejidad

Construir SA + LCP cuesta  $O(n \log n)$  o  $O(n)$ . La pila monotonamente decreciente recorre LCP en  $O(n)$ . Memoria  $O(n)$ .

## 10 Substring periodica

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/695758/problem/I>

### Resumen

Una cadena es periodica si se puede describir por repeticiones de un patron mas corto. Para substrings, se necesita verificar periodicidad de muchas porciones de un texto.

### Teoria desde cero

Una cadena  $X$  de longitud  $m$  tiene periodo  $p$  si:

$$X[i] = X[i + p] \quad \text{para todo } 0 \leq i < m - p.$$

Si  $p$  divide a  $m$  y el periodo cubre todo,  $X$  es repeticion de un bloque de longitud  $p$ . Herramientas utiles: prefix function, Z-function y LCP entre sufijos.

### Estructura

Para muchas comparaciones de substrings, usa suffix array + LCP + RMQ. Eso permite responder en  $O(1)$  si dos substrings largas son iguales: basta con ver si el LCP de sus posiciones alcanza la longitud requerida.

### Algoritmo conceptual

Para probar si una substring tiene periodo  $p$ , verifica que su prefijo de longitud  $m - p$  coincide con la substring desplazada  $p$  posiciones. Eso se responde con LCP:

$$\text{lcp}(l, l + p) \geq m - p.$$

Para buscar periodos, se prueban candidatos segun divisores o segun estructura del problema.

### Complejidad

Con preprocesamiento SA + LCP + RMQ, cada prueba de igualdad de substrings cuesta  $O(1)$ . La complejidad total depende de cuantos periodos candidatos se prueben. Este es de los mas avanzados; asegura primero SA, LCP y RMQ.

## 11 Tree and Queries

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/697527/problem/A>

### Resumen

Cada nodo tiene un color. Cada consulta pregunta, en el subarbol de un nodo  $v$ , cuantos colores aparecen al menos  $k$  veces.

### Teoria desde cero

Euler tour convierte el subarbol de  $v$  en un intervalo, pero la pregunta no es un simple minimo/suma: pide frecuencias de colores dentro del subarbol. La tecnica natural es **DSU on tree** o **small-to-large**.

### Estructura

Para cada color  $c$ , guarda  $cntColor[c]$ . Para responder “cuantos colores tienen frecuencia al menos  $k$ ”, guarda tambien informacion por frecuencia, por ejemplo un Fenwick sobre  $cntFreq[f]$ , donde  $cntFreq[f]$  es cuantos colores aparecen exactamente  $f$  veces.

### Algoritmo

Elige en cada nodo un hijo pesado: el de mayor subarbol. Procesa hijos ligeros y borra su informacion. Procesa el hijo pesado y conserva su informacion. Luego agrega manualmente todos los nodos de los hijos ligeros y el nodo actual. En ese momento la bolsa contiene exactamente el subarbol actual, asi que respondes sus consultas.

### Correctitud

Los subarboles de los hijos de  $v$  son disjuntos y junto con  $v$  forman todo el subarbol de  $v$ . Al conservar el hijo pesado y agregar los ligeros, la bolsa queda exactamente igual al subarbol completo. Como las consultas se responden en ese instante, las frecuencias son correctas.

### Complejidad

Cada nodo se re-agrega solo cuando pertenece a un subarbol ligero. En una ruta raiz-hoja hay  $O(\log n)$  aristas ligeras, porque al subir por una ligera el tamano se duplica. Cota segura:  $O(n \log n)$  mas el costo de updates de frecuencias.

## 12 Arpa's letter-marked tree

Link. <https://codeforces.com/group/Amk1RC8lhZ/contest/697527/problem/B>

### Resumen

Problema de arbol con letras donde se razona si caminos o conjuntos de letras pueden formar palindromos.

### Teoria desde cero

Una multiconjunto de letras puede reordenarse como palindromo si a lo mucho una letra aparece cantidad impar de veces. Esta condicion se representa con una mascara de bits: bit  $j = 1$  si la letra  $j$  aparece impar cantidad de veces. Una mascara es valida para palindromo si tiene cero o un bit prendido.

### Estructura

Define  $maskRoot[v]$  como xor de letras desde la raiz hasta  $v$ . La mascara del camino  $u \rightsquigarrow v$  se obtiene combinando mascaras de prefijo y ajustando el LCA. Conceptualmente:

$$mask(u, v) = maskRoot[u] \oplus maskRoot[v] \oplus letter[lca(u, v)].$$

Segun si las letras estan en vertices o aristas, el ajuste cambia, pero la idea es xor de prefijos.

### Algoritmo

Para contar pares o caminos validos, se mantiene un contador de mascaras. Una mascara  $x$  puede emparejarse con:

$$x \text{ o } x \oplus 2^b$$

para cualquier bit  $b$ . Esto cubre cero o una paridad impar. En subarboles, se combina con DSU on tree para mantener conteos de mascaras eficientemente.

### Correctitud

El xor cancela letras que aparecen dos veces. Por tanto la mascara resultante codifica exactamente que letras aparecen impar cantidad de veces en el camino. La condicion de palindromo es equivalente a que esa mascara tenga a lo mucho un bit encendido.

### Complejidad

Con alfabeto pequeno, revisar todas las mascaras que difieren en un bit cuesta  $O(\Sigma)$  por insercion/consulta. Con DSU on tree, cota segura  $O(n \log n \cdot \Sigma)$ .

## 13 Laminar Family

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/697527/problem/C>

### Resumen

Una familia laminar es una coleccion de conjuntos donde dos conjuntos cualesquiera son disjuntos o uno contiene al otro. En arboles, los subarboles tienen naturalmente esta forma.

### Teoria desde cero

Haz un DFS y asigna tiempos  $\text{tin}[v]$  y  $\text{tout}[v]$ . El subarbol de  $v$  se vuelve el intervalo:

$$[\text{tin}[v], \text{tout}[v]].$$

Un nodo  $u$  es ancestro de  $v$  si:

$$\text{tin}[u] \leq \text{tin}[v] \leq \text{tout}[u].$$

Dos subarboles en un arbol no se cruzan parcialmente: o son disjuntos, o uno contiene al otro.

### Algoritmo conceptual

Representa cada conjunto/subarbol por su intervalo Euler. Para verificar si una familia es laminar, ordena intervalos por inicio y usa una pila de intervalos activos. Cuando entra un intervalo nuevo, debe estar contenido en el ultimo activo o empezar despues de que los activos incompatibles terminen. Si hay interseccion parcial, no es laminar.

### Correctitud

La transformacion DFS preserva inclusion de subarboles como inclusion de intervalos. Las unicas relaciones validas en una familia laminar son inclusion o disjuncion. Por eso detectar cruces parciales entre intervalos es equivalente a detectar violaciones de laminaridad.

### Complejidad

DFS  $O(n)$ . Ordenamiento  $O(m \log m)$  si hay  $m$  conjuntos. La verificacion con pila es  $O(m)$ .

## 14 Xenia and Tree

Link. <https://codeforces.com/group/Amk1RC8lhZ/contest/697527/problem/D>

### Resumen

Problema clasico: hay vertices rojos en un arbol. Se actualiza un vertice a rojo y se pregunta la distancia al rojo mas cercano.

### Teoria desde cero

Si recalculas BFS desde cada consulta, es demasiado lento. La estructura natural es **centroid decomposition**. Un centroeide es un nodo cuya eliminacion parte el arbol en componentes de tamano a lo mucho la mitad. Al descomponer recursivamente, cada nodo queda asociado a  $O(\log n)$  centroides ancestros.

### Estructura

Para cada centroeide  $c$ , guarda:

$$best[c] = \min \text{dist}(c, r)$$

entre todos los rojos  $r$  ya activados. Para cada nodo  $v$ , guarda la lista de centroides ancestros y la distancia de  $v$  a cada uno.

### Algoritmo

Al pintar  $v$  rojo, para cada centroeide ancestro  $c$  de  $v$ , actualiza:

$$best[c] = \min(best[c], \text{dist}(v, c)).$$

Para consultar  $u$ , calcula:

$$\min_c best[c] + \text{dist}(u, c),$$

donde  $c$  recorre los centroides ancestros de  $u$ .

### Correctitud

Para cualquier rojo  $r$  y consulta  $u$ , existe un nivel de la descomposicion de centroides donde los caminos desde  $u$  y  $r$  se encuentran representados por un centroeide comun  $c$ . La distancia via ese centroeide se considera como  $best[c] + \text{dist}(u, c)$ . Al tomar minimo sobre todos los centroides ancestros, se captura el rojo mas cercano.

### Complejidad

Cada nodo tiene  $O(\log n)$  centroides ancestros. Update y query cuestan  $O(\log n)$  si las distancias a centroides estan precomputadas. Construccion  $O(n \log n)$ .



## 15 Beauty and The Tree

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/697527/problem/E>

### Resumen

Problema de arbol mas avanzado, asociado a DP/probabilidad o conteos sobre subarboles. No es tan central como Tree and Queries o Xenia, pero entrena razonamiento de arboles.

### Teoria desde cero

Para cualquier DP en arbol, primero se enraiza. Luego se define que informacion calcula cada nodo a partir de sus hijos. Si la respuesta de un nodo depende solo de su subarbol, un DFS postorden basta. Si depende de todo el arbol visto desde ese nodo, se necesita rerooting.

### Estructura

Guarda:

- padre y profundidad;
- tamano de subarbol;
- valores DP que suben desde hijos;
- si hace falta rerooting, valores prefijo/sufijo de contribuciones de hijos.

### Algoritmo conceptual

Primero calcula informacion local de subarboles. Luego, si la query pide respuesta global por nodo, realiza una segunda pasada para pasar contribuciones del padre hacia hijos. Este patron se llama rerooting DP.

### Correctitud

El postorden es correcto porque al procesar un nodo ya se conocen todos sus hijos. El rerooting es correcto porque separa la contribucion del hijo actual de la contribucion del resto del arbol, permitiendo reconstruir la respuesta como si cada hijo fuera raiz.

### Complejidad

Normalmente  $O(n)$  o  $O(n \log n)$  segun la combinacion de estados. Para examen, lo importante es justificar que cada arista se procesa cantidad constante de veces.

## 16 Connect and Disconnect

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/698987/problem/A>

### Resumen

Se agregan y eliminan aristas, y se hacen consultas de conectividad. Como hay eliminaciones, DSU normal no basta.

### Teoria desde cero

DSU responde conectividad incremental: solo agregar aristas. Si borras aristas, DSU no sabe separar componentes. La solución offline convierte cada arista en intervalos de vida. Luego procesa el tiempo con un segment tree.

### Estructura

Usa:

- mapa de aristas activas para saber inicio de intervalo;
- segment tree sobre tiempos  $1..q$ ;
- DSU con rollback, sin path compression;
- pila de cambios para deshacer unions.

### Algoritmo

Cada arista activa en  $[l, r)$  se inserta en  $O(\log q)$  nodos del segment tree temporal. Recorre el segment tree en DFS. Al entrar a un nodo, une todas sus aristas. Si es hoja, responde las consultas de ese tiempo. Al salir, revierte el DSU al estado anterior.

### Correctitud

En una hoja  $t$ , el camino desde la raíz del segment tree hasta esa hoja contiene exactamente todos los nodos temporales cuyos intervalos cubren  $t$ . Por construcción, esos nodos contienen todas y solo las aristas activas en  $t$ . Por eso el DSU representa el grafo correcto para responder conectividad.

### Complejidad

Cada intervalo va a  $O(\log q)$  nodos. Cada union/find cuesta  $O(\log n)$  sin compresión o casi constante en práctica con union by size. Cota segura:

$$O((q + m) \log q \log n).$$

## 17 Addition on Segments

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/698987/problem/B>

### Resumen

Problema de operaciones activas en segmentos. Aunque no sea grafo, entrena el mismo patron mental que offline dynamic connectivity: objetos que viven en intervalos.

### Teoria desde cero

Si una operacion afecta un rango de tiempos o posiciones, puedes descomponer ese rango en nodos de un segment tree. Asi evitas aplicar la operacion uno por uno en cada punto.

### Estructura

Segment tree sobre el eje relevante: tiempo, posiciones o indices. Cada nodo guarda las operaciones que cubren completamente su intervalo. Si el problema requiere deshacer, la estructura interna debe soportar rollback.

### Algoritmo

Para cada operacion de adiccion sobre  $[l, r]$ , insertala en el segment tree en  $O(\log n)$  nodos. Recorre el arbol manteniendo el estado acumulado. En hojas respondes el valor final o la consulta correspondiente. Al volver, deshaces lo que agregaste si el estado no es persistente.

### Correctitud

Todo punto  $x$  pertenece exactamente a los nodos del camino raiz-hoja que lo cubren. Una operacion insertada en nodos que particionan  $[l, r]$  estara activa exactamente para los puntos de ese rango. Por eso al llegar a una hoja, el estado incluye todas y solo las operaciones relevantes.

### Complejidad

Cada intervalo se guarda en  $O(\log n)$  nodos. Si cada aplicar/deshacer cuesta  $T$ , el total es  $O(Q \log n \cdot T)$ .

## 18 Visitas

**Link.** <https://codeforces.com/group/Amk1RC8lhZ/contest/699336/problem/A>

### Resumen

Es el problema tipo *Milk Visits*. Hay un arbol de granjas; cada nodo tiene un tipo de vaca. Cada consulta da dos nodos  $a, b$  y un tipo  $c$ . Se pregunta si en el camino entre  $a$  y  $b$  existe al menos un nodo de tipo  $c$ .

### Teoria desde cero

Una solucion general con HLD podria consultar colores en caminos, pero seria mas pesada. La observacion clasica de Milk Visits es por componentes de igual color. Si dos nodos  $a$  y  $b$  estan en la misma componente maximal de nodos del mismo tipo, entonces todo el camino entre ellos tiene ese mismo tipo. Si el tipo preferido  $c$  es distinto, no habra ningun  $c$  en ese camino.

### Estructura

Comprime componentes conectadas de nodos con el mismo tipo. Asigna a cada nodo un id de componente. Para una consulta  $(a, b, c)$ :

- si  $comp[a] \neq comp[b]$ , el camino cruza al menos dos componentes de tipos distintos, y se puede satisfacer cualquier consulta cuyo criterio sea la version clasica de Milk Visits;
- si  $comp[a] = comp[b]$ , todo el camino tiene el tipo de esa componente, y la respuesta es si ese tipo coincide con  $c$ .

### Correctitud

Una componente maximal de igual tipo contiene exactamente nodos conectados por caminos de ese tipo. Si  $a$  y  $b$  estan en la misma componente, el camino unico entre ellos no sale de la componente, por tanto todos sus nodos tienen el mismo tipo. Si estan en componentes distintas, el camino cambia de tipo al menos una vez; en la version clasica, eso garantiza satisfaccion salvo cuando el camino entero es de un unico tipo distinto al pedido.

### Complejidad

DFS para construir componentes:  $O(n)$ . Cada consulta:  $O(1)$ . Memoria:  $O(n)$ .

## 19 Distancia unitaria

Link. <https://codeforces.com/group/Amk1RC8lhZ/contest/694842/problem/A>

### Resumen

Se da un arbol enraizado en 1. Cada consulta da un conjunto de vertices. Se pregunta si existe un camino desde la raiz hasta algun nodo  $u$  tal que cada vertice de la consulta este en ese camino o a distancia 1 de algun vertice del camino.

### Teoria desde cero

Si un vertice  $v$  esta a distancia 1 de un camino raiz- $u$ , entonces su padre puede estar en el camino. Por eso se transforma cada vertice consultado en su padre (excepto la raiz, que se queda igual). Luego la pregunta se vuelve: estan todos esos vertices transformados sobre un mismo camino desde la raiz?

### Estructura

Preprocesa:

- profundidad;
- padre inmediato;
- tiempos Euler tin, tout para ancestro;
- opcionalmente LCA.

Un nodo  $x$  es ancestro de  $y$  si:

$$\text{tin}[x] \leq \text{tin}[y] \leq \text{tout}[x].$$

### Algoritmo

Para cada consulta, reemplaza cada vertice  $v$  por  $\text{parent}[v]$  si  $v \neq 1$ . Elige el vertice transformado mas profundo, llamalo *deep*. La respuesta es YES si todos los vertices transformados son ancestros de *deep*. Si alguno no lo es, no existe un unico camino raiz- $u$  que los cubra.

### Correctitud

Despues de subir cada vertice a su padre, la condicion de estar en el camino o a distancia 1 se convierte en estar sobre el camino. Si todos los transformados estan en un camino raiz- $u$ , el mas profundo de ellos puede actuar como extremo o estar antes del extremo, y todos los demas deben ser ancestros de el. Si alguno no es ancestro del mas profundo, se ramifica hacia otro lado y ningun camino simple desde la raiz puede contenerlos a todos.

### Complejidad

DFS inicial  $O(n)$ . Cada consulta cuesta proporcional a su tamano. Como la suma de tamanos de consultas esta acotada, el total es  $O(n + \sum k_i)$ .

## 20 Imperial roads

**Link.** <https://codeforces.com/gym/101889/problem/I>

### Resumen

Se tiene un grafo ponderado. Para cada consulta, una arista debe estar incluida en el árbol generador. Se pide el mínimo peso posible.

### Teoría desde cero

Primero construye un MST con Kruskal. Si fuerzas una arista que no está en el MST, al agregarla aparece un ciclo. Para volver a tener un árbol, debes eliminar una arista de ese ciclo. Para minimizar el costo, eliminas la arista más pesada del camino dentro del MST.

### Estructura

Necesitas responder máximo peso de arista en el camino entre dos nodos del MST. Usa LCA con binary lifting aumentado:

- $up[v][j]$ : ancestro  $2^j$  de  $v$ ;
- $mx[v][j]$ : máximo peso en el salto de  $v$  a  $up[v][j]$ .

### Algoritmo

Sea  $W$  el peso del MST. Para consulta  $(u, v, w)$ :

$$ans = W + w - \text{maxEdge}(u, v).$$

Para calcular  $\text{maxEdge}$ , iguala profundidades subiendo el nodo más profundo y acumulando máximos; luego sube ambos nodos juntos hasta llegar al LCA.

### Correctitud

Agregar  $(u, v, w)$  al MST crea un ciclo único. Todo árbol que incluya esa arista y use las demás del MST salvo una debe eliminar una arista del camino  $u - v$ . Eliminar la más pesada minimiza el nuevo peso. La propiedad del ciclo de MST respalda que la arista más pesada del ciclo es la candidata natural a remover.

### Complejidad

Kruskal  $O(m \log m)$ . Preprocesamiento LCA  $O(n \log n)$ . Cada consulta  $O(\log n)$ . Memoria  $O(n \log n)$ .

## 21 Fools and Roads

**Link.** <https://codeforces.com/problemset/problem/191/C>

### Resumen

Se da un árbol y muchas rutas entre pares de vertices. Para cada arista, se pide cuantas rutas pasan por ella.

### Teoria desde cero

Actualizar cada camino arista por arista seria lento. Como todas las rutas se conocen antes de responder, se usa diferencia en árbol. Es la version arborea de un difference array.

### Algoritmo

Para cada ruta  $(u, v)$ :

$$\text{delta}[u] ++, \quad \text{delta}[v] ++, \quad \text{delta}[\text{lca}(u, v)] - = 2.$$

Luego haces DFS postorden. Para cada hijo  $x$ , sumas sus acumulados hacia el padre. El valor final acumulado en  $x$  es la cantidad de rutas que pasan por la arista  $(\text{parent}[x], x)$ .

### Correctitud

Una ruta  $u - v$  sube desde  $u$  hasta  $a = \text{lca}(u, v)$  y baja hasta  $v$ . Los  $+1$  en  $u$  y  $v$  hacen que al acumular suban dos contribuciones. Restar 2 en el LCA cancela ambas antes de que pasen por encima del LCA. Asi se marcan exactamente las aristas de la ruta y ninguna otra. Por linealidad, funciona para todas las rutas juntas.

### Complejidad

Preprocesar LCA  $O(n \log n)$ . Cada ruta  $O(\log n)$ . DFS final  $O(n)$ . Total  $O((n + q) \log n)$ .

## 22 Frequency of String

**Link.** <https://codeforces.com/problemset/problem/963/D>

### Resumen

Para un texto fijo y consultas  $(k, P)$ , se pide la minima longitud de una subcadena que contenga al patron  $P$  al menos  $k$  veces.

### Teoria desde cero

Si conoces todas las posiciones donde aparece  $P$ :

$$p_1 < p_2 < \dots < p_t,$$

entonces una ventana con  $k$  ocurrencias consecutivas desde  $p_i$  hasta  $p_{i+k-1}$  tiene longitud:

$$p_{i+k-1} - p_i + |P|.$$

La respuesta es el minimo sobre  $i$ .

### Estructura

Para muchos patrones, usa Aho-Corasick: insertas todos los patrones en un trie, construyes enlaces de fallo y recorres el texto una vez para recolectar ocurrencias. Alternativamente, suffix array permite encontrar el intervalo de sufijos que empiezan con  $P$ , pero luego hay que ordenar posiciones por aparicion en el texto.

### Correctitud

Cualquier subcadena que contiene  $k$  ocurrencias tiene una primera ocurrencia y una  $k$ -esima ocurrencia. Para contener la  $k$ -esima debe terminar al menos en  $p_{i+k-1} + |P| - 1$ . Tomar exactamente desde  $p_i$  hasta ese final alcanza esa longitud. Por tanto revisar ocurrencias consecutivas es suficiente y optimo.

### Complejidad

Aho-Corasick se construye en  $O(L)$  o  $O(L|\Sigma|)$  segun transiciones, donde  $L$  es suma de longitudes. El recorrido del texto es  $O(n)$  mas ocurrencias reportadas. Para cada patron, calcular ventanas cuesta lineal en su numero de ocurrencias.



## 23 Yosupo - Dynamic Graph Vertex Add Component Sum

**Link.** [https://judge.yosupo.jp/problem/dynamic\\_graph\\_vertex\\_add\\_component\\_sum](https://judge.yosupo.jp/problem/dynamic_graph_vertex_add_component_sum)

### Resumen

Grafo dinamico con pesos en vertices. Hay cambios de aristas, incrementos de valor en vertices y consultas de suma de la componente de un vertice.

### Teoria desde cero

El problema combina conectividad dinamica con agregados de componente. Offline, la herramienta central es segment tree sobre tiempo + DSU rollback. El DSU no solo guarda padre/tamano, tambien suma total de cada componente.

### Algoritmo

Convierte cada arista en intervalos de vida. Inserta cada intervalo en el segment tree temporal. Recorre en DFS: al entrar, unes aristas y actualizas sumas; en hojas, respondes la suma de la componente del vertice consultado; al salir, reviertes cambios.

### Correctitud

En la hoja  $t$ , el DSU contiene exactamente las aristas activas en  $t$ . Como cada union actualiza la suma de la nueva componente y rollback restaura el estado anterior, el agregado guardado en la raiz es exactamente la suma de la componente actual.

### Complejidad

Cada intervalo aparece en  $O(\log q)$  nodos. Cota segura  $O((q + m) \log q \log n)$  sin path compression. Memoria  $O((q + m) \log q)$  para intervalos.

## 24 Yosupo - Dynamic Tree Subtree Add Subtree Sum

**Link.** [https://judge.yosupo.jp/problem/dynamic\\_tree\\_subtree\\_add\\_subtree\\_sum](https://judge.yosupo.jp/problem/dynamic_tree_subtree_add_subtree_sum)

### Resumen

Arbol dinamico con link/cut, update de suma en subarbol y query de suma de subarbol.

### Teoria desde cero

HLD clasico no basta porque el arbol cambia. Una estructura natural es Euler-Tour Tree: representar cada arbol como una secuencia Euler guardada en un BST balanceado implicito. Cortar y enlazar se vuelven operaciones de split y merge sobre secuencias.

### Estructura

Cada nodo del BST guarda:

- tamano del segmento;
- suma del segmento;
- lazy add pendiente;
- punteros para split/merge.

El subarbol de un nodo se representa como un intervalo bajo una orientacion de raiz adecuada.

### Algoritmo

Para update de subarbol, aisla el intervalo del subarbol con splits, aplica lazy add y vuelve a unir. Para query, aisla el intervalo, lee la suma agregada y vuelve a unir. Link y cut modifican la secuencia Euler con split/merge.

### Correctitud

El invariante es que la secuencia Euler representa exactamente el arbol actual. Los splits no cambian el orden interno, solo aislan intervalos. Como el subarbol corresponde a un intervalo, lazy add afecta exactamente sus vertices, y la suma agregada corresponde exactamente al subarbol.

### Complejidad

Cada split/merge cuesta  $O(\log n)$  esperado/amortizado. Cada operacion usa cantidad constante de splits/merges:  $O(\log n)$ .

## 25 SPOJ - Dynamic Tree Connectivity

**Link.** <https://www.spoj.com/problems/DYNACON1/>

### Resumen

Bosque dinamico con operaciones link, cut y connected. Se pregunta si dos vertices estan en la misma componente.

### Teoria desde cero

Si solo hubiera link, DSU bastaria. Con cut, DSU no puede separar. Online se usan Link-Cut Trees o Euler-Tour Trees. Offline tambien se puede usar segment tree sobre tiempo + DSU rollback.

### Opcion online

Link-Cut Tree mantiene preferred paths con splay trees. Permite link, cut y find-root en  $O(\log n)$  amortizado. Dos nodos estan conectados si tienen la misma raiz representada.

### Opcion offline

Cada arista tiene intervalos de vida entre link y cut. Se inserta en segment tree temporal y se recorre con DSU rollback. En hojas, connected se responde comparando representantes.

### Correctitud

En Link-Cut/Euler-Tour Trees, el invariante es que la estructura representa el bosque actual. En rollback, el invariante es temporal: al llegar a la hoja  $t$ , el DSU contiene exactamente las aristas activas en  $t$ .

### Complejidad

Online:  $O(\log n)$  amortizado por operacion. Offline: cada intervalo en  $O(\log q)$  nodos, total aproximadamente  $O(q \log q \log n)$  con DSU rollback sin compresion.

## 26 Resumen final

### Lo que debes dominar si no sabes nada

1. Sparse table para RMQ.
2. Suffix array y LCP.
3. Búsqueda de patron como intervalo en suffix array.
4. Formula de subcadenas distintas.
5. LCP + RMQ para comparar subcadenas.
6. Euler tour, LCA y ancestros por tiempos.
7. DSU on tree para consultas de subarbol.
8. Mascaras xor para paridad en caminos.
9. Centroid decomposition para distancia a nodo activado.
10. Segment tree sobre tiempo + DSU rollback para dynamic connectivity.
11. MST + max edge path para Imperial Roads.
12. Euler-Tour Trees/Link-Cut Trees como estructuras dinamicas de arbol.

### Formato de respuesta de examen

Para cada problema escribe:

1. Que estructura uso y por que.
2. Que guarda la estructura.
3. Como se responde una consulta/update.
4. Invariante de correctitud.
5. Complejidad de preprocesamiento, query/update y memoria.